

Fortran OOP and Python

Rohit Goswami · 2022-05-09 · python · fortran · numpy · f2py · ramblings

Explorations of object oriented Fortran with `bind(c)` derived types for representations generated by F2PY

Background

Derived types are easily one of the most visible of the modern Fortran (post-F90) features and are central to object oriented programming paradigms in Fortran.

For those new to the language, a rough guide to some terminology:

Fortran	Closest C/C++ equivalent
derived type	struct
extends type	inherited class
final	destructor
not standard conforming undefined behaviour	

Only the first of these are actually covered in terms of interoperability with `C` as of the [F2018 draft standard](#).

For F2PY within `numpy`, however, although there are no direct equivalents in `C` for the more OOP specific derived types, there are plenty of equivalent structures in `Python`. This post will bridge Fortran derived types to their logically equivalent Python class definitions.

Premise

As in [the previous posts](#), we will be interested in a `bind(c)` derived type.

```
type :: cartesian
  real(c_double) :: x, y, z
end type cartesian
```

Along with a simple function which uses an instance of such a derived type.

```
subroutine unit_step(arg) bind(c)
  type(cartesian), intent(inout) :: arg
  arg%x = arg%x + 1
  arg%y = arg%y + 1
  arg%z = arg%z + 1
end subroutine unit_step
```

To make the discussion and conversion into Python more concrete, we will start with an overview of possible representations.

Representations

The elements of the Python programming language which can be best used to emulate a derived type are (in order of relative complexity):

Lists

In theory, a record type can be represented and constructed from a list, with the members of the type specified in order of appearance or lexicographically. However, this is a lossy, incomplete representation as it would remove the ability to refer to and operate on the member variables by name.

Dictionaries

These have the benefit of being relatively simple to implement, and retain member addressable semantics.

Named Tuples

As immutable data structures with labels, these make good return types, however, they are poor representations for objects which may be modified in place (e.g. in a Fortran `subroutine`).

Dataclasses

These make sense from a logical perspective, however, they are hard to programmatically generate and use in C-extensions, limiting their utility for inter-operability with Fortran.

Classes

The most general and arguably the most flexible construct is to generate code for a `PyTypeObject`, which can be fleshed out to contain both attributes, as well as memory allocation rules.

Of the representations described, the first approximation covered in

[a previous post](#)

() was the `dictionary`. A major concern [^fn:1] is that of lineage and extensibility; `python` users would expect that a dictionary can be extended with keys, which would not make much sense if the dictionary is meant to represent a derived type. Additionally, it is difficult to programmatically ascertain which dictionaries are meant to be interoperable with Fortran functions or subroutines. Note that the dictionary representation code is also [present here](#) with usage examples.

The rest of this post will enumerate the construction and usage of a `bind(c)` derived type represented as a Python class.

Pythonic Equivalent

For a more pedagogical understanding, the derived type will be emulated (from within a Python-C extension) by the following class definition:

```

class fakecart:
    def __new__(cls, *args, **kwargs):
        obj = object.__new__(cls)
        obj.x = 0
        obj.y = 0
        obj.z = 0
        return obj

    def __init__(self, x, y, z):
        self.x = x
        self.y = y
        self.z = z

    def __repr__(self):
        return f"fakecart(x: {self.x}, y: {self.y}, z: {self.z})"

    def unitstep(self):
        self.x = self.x + 1
        self.y = self.y + 1
        self.z = self.z + 1

```

Note that apart from the members which mimic the those in the derived type, for illustration's sake we will implement the `unit_step` free function as a member function.

C-Python-Fortran Implementation

For `bind(c)` types, we will focus on implementing the member data in terms of `C` structs, and rely on `bind(c)` to ensure that symbols are mangled in a compatible manner, i.e. we will not need to hardcode any compiler specific name mangling conventions for the functions we call from Fortran. The complete Fortran code of interest is:

```

! vec.f90
module vec
    use, intrinsic :: iso_c_binding
    implicit none

    type :: cartesian
        real(c_double) :: x, y, z
    end type cartesian

    contains
        subroutine unit_step(arg) bind(c)
            type(cartesian), intent(inout) :: arg
            arg%x = arg%x + 1
            arg%y = arg%y + 1
            arg%z = arg%z + 1
        end subroutine unit_step
end module vec

```

For a source to source translation from the `Python` code, we will start with a header file containing the basic C-Fortran structures and function declarations:

```
// vecf.h
#ifdef VECF_H_
#define VECF_H_

#include<stdio.h>
#include<stdlib.h>

// bind(c) compatible struct declaration
typedef struct {
    double x, y, z;
} cartesian;

// In Fortran
void unit_step(cartesian *arg);

#endif // VECF_H_
```

Now for the class declaration itself.

Instance Variables and Initialization

Rather than declaring `double` variables in the Python class, we will opt to use the `struct` defined earlier.

```
// CPython compatible declaration
typedef struct {
    PyObject_HEAD          /* Do not move */
    cartesian ccart; /* totally interoperable */
} pycart;
```

This means we can initialize an object of this class by writing the equivalent of the `__init__()` as:

```
static int pycart_init(pycart *self, PyObject *args, PyObject *kwargs) {
    static char *kwlist[] = {"x", "y", "x", NULL};
    if (!PyArg_ParseTupleAndKeywords(args, kwargs, "|ddd", kwlist, &self->ccart.x,
                                     &self->ccart.y, &self->ccart.z))
        return -1;
    return 0;
}
```

Finally we have to declare the instance variables to be exposed from the class.

```
static PyMemberDef pycart_members[] = {
    {"x", T_DOUBLE, offsetof(pycart, ccart.x), 0, "x coordinate"},
    {"y", T_DOUBLE, offsetof(pycart, ccart.y), 0, "y coordinate"},
    {"z", T_DOUBLE, offsetof(pycart, ccart.z), 0, "z coordinate"},
    {NULL} /* Sentinel */
};
```

Note that in each of the snippets above, we leverage the `cartesian` structure.

Class Methods

The heavy lifting of the function itself is in Fortran, while the `C` struct can be used inter-operably with a derived type, leaving us with an extremely light definition:

```
static PyObject *pycart_unitstep(pycart *self) {
    // Call Fortran on the internal representation
    unit_step(&self->ccart);
    Py_RETURN_NONE;
}
```

We will also define a `__repr()` function for ease of interactive use:

```
static PyObject *pycart_repr(pycart *self) {
    char x[50], y[50], z[50];
    sprintf(x, "%f", self->ccart.x);
    sprintf(y, "%f", self->ccart.y);
    sprintf(z, "%f", self->ccart.z);
    PyObject *repr = PyUnicode_FromFormat("pycart(x: %s, y: %s, z: %s)", x, y, z);
    return repr;
}
```

Finally we can register the methods.

```
static PyMethodDef pycart_methods[] = {
    {"unitstep", (PyCFunction)pycart_unitstep, METH_NOARGS,
     "Modify the class members with the Fortran unit_step function"},
    {NULL} /* Sentinel */
};
```

Before rounding out our definition:

```
// Definition of type
static PyTypeObject t_pycart = {
    PyVarObject_HEAD_INIT(NULL, 0) /* tp_head */
    .tp_name = "pycart.pycart",
    .tp_doc = PyDoc_STR("One coordinate (x, y, z)"),
    .tp_basicsize = sizeof(pycart),
    .tp_itemsize = 0,
    .tp_dealloc = pycart_dealloc,
    .tp_repr = pycart_repr,
    .tp_getattro = PyObject_GenericGetAttr,
    .tp_setattro = PyObject_GenericSetAttr,
    .tp_flags = Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE,
    .tp_members = pycart_members,
    .tp_methods = pycart_methods,
    .tp_init = pycart_init,
    .tp_alloc = PyType_GenericAlloc,
    .tp_new = PyType_GenericNew,
    .tp_free = PyObject_Del,
};
```

Where we have additionally defined and bound machinery to declare `__new__()`.

```

static void pycart_dealloc(pycart *self) {
    // Also free the pointer here
    Py_TYPE(self)->tp_free((PyObject *)self);
}

static PyObject *pycart_new(PyTypeObject *type, PyObject *args,
                           PyObject *kwargs) {
    // Allocate and create the cartesian object here
    pycart *self;
    self = (pycart *)type->tp_alloc(type, 0);
    if (self != NULL) {
        self->ccart.x = 0;
        self->ccart.y = 0;
        self->ccart.z = 0;
    }
    return (PyObject *)self;
}

```

Boilerplate and Builds

We round off the extension by adding some doc-strings and also setting up the module itself. The code for the module itself is then:

```

#ifndef PY_SSIZE_T_CLEAN
#define PY_SSIZE_T_CLEAN
#include <stdio.h>
#endif /* PY_SSIZE_T_CLEAN */

#include "Python.h"
#include "structmember.h"

// Fortran-C stuff
#include "vecf.h"

// Python-C stuff

// CPython compatible declaration
typedef struct {
    PyObject_HEAD      /* Do not move */
    cartesian ccart; /* totally interoperable */
} pycart;

// Almost everything in the class must be static
static void pycart_dealloc(pycart *self) {
    Py_TYPE(self)->tp_free((PyObject *)self);
}

static PyObject *pycart_new(PyTypeObject *type, PyObject *args,
                             PyObject *kwargs) {
    pycart *self;
    self = (pycart *)type->tp_alloc(type, 0);
    if (self != NULL) {
        self->ccart.x = 0;
        self->ccart.y = 0;
        self->ccart.z = 0;
    }
    return (PyObject *)self;
}

static int pycart_init(pycart *self, PyObject *args, PyObject *kwargs) {
    static char *kwlist[] = {"x", "y", "z", NULL};
    if (!PyArg_ParseTupleAndKeywords(args, kwargs, "|ddd", kwlist, &self->ccart.x,
                                     &self->ccart.y, &self->ccart.z))
        return -1;
    return 0;
}

static PyMemberDef pycart_members[] = {
    {"x", T_DOUBLE, offsetof(pycart, ccart.x), 0, "x coordinate"},
    {"y", T_DOUBLE, offsetof(pycart, ccart.y), 0, "y coordinate"},
    {"z", T_DOUBLE, offsetof(pycart, ccart.z), 0, "z coordinate"},
    {NULL} /* Sentinel */
};

static PyObject *pycart_repr(pycart *self) {
    char x[50], y[50], z[50];
    sprintf(x, "%f", self->ccart.x);
    sprintf(y, "%f", self->ccart.y);
    sprintf(z, "%f", self->ccart.z);
    PyObject *repr = PyUnicode_FromFormat("pycart(x: %s, y: %s, z: %s)", x, y, z);
    return repr;
}

static PyObject *pycart_unitstep(pycart *self) {
    // Call Fortran on the internal representation
    unit_step(&self->ccart);
    Py_RETURN_NONE;
}

```

```

static PyMethodDef pycart_methods[] = {
    {"unitstep", (PyCFunction)pycart_unitstep, METH_NOARGS,
     "Modify the class members with the Fortran unit_step function"},
    {NULL} /* Sentinel */
};

// Definition of type
static PyTypeObject t_pycart = {
    PyVarObject_HEAD_INIT(NULL, 0) /* tp_head */
    .tp_name = "pycart.pycart",
    .tp_doc = PyDoc_STR("One coordinate (x, y, z)"),
    .tp_basicsize = sizeof(pycart),
    .tp_itemsize = 0,
    .tp_dealloc = pycart_dealloc,
    .tp_repr = pycart_repr,
    .tp_getattro = PyObject_GenericGetAttr,
    .tp_setattro = PyObject_GenericSetAttr,
    .tp_flags = Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE,
    .tp_members = pycart_members,
    .tp_methods = pycart_methods,
    .tp_init = pycart_init,          /* tp_init */
    .tp_alloc = PyType_GenericAlloc, /* tp_alloc */
    .tp_new = PyType_GenericNew,     /* tp_new */
    .tp_free = PyObject_Del,         /* tp_free */
    // Add an attribute to show this is from Fortran
};

static char pycart_docs[] = "pycart: data type with x,y,z elements\n";

static PyModuleDef pycartmodule = {PyModuleDef_HEAD_INIT, .m_name = "pycart",
                                   .m_doc = pycart_docs, .m_size = -1};

PyMODINIT_FUNC PyInit_pycart(void) {
    PyObject *this_module;
    if (PyType_Ready(&t_pycart))
        return NULL;
    this_module = PyModule_Create(&pycartmodule);
    if (this_module == NULL) {
        return NULL;
    }
    Py_INCREF(&t_pycart);
    if (PyModule_AddObject(this_module, "pycart", (PyObject *)&t_pycart) < 0) {
        Py_DECREF(&t_pycart);
        Py_DECREF(this_module);
        return NULL;
    }
    return this_module;
}

```

Which is coupled with a `meson.build` file:


```

project('pyclass_bindc', 'c', 'fortran',
        version : '0.1',
        default_options : ['warning_level=2',
                           'buildtype=debug',
                           'debug=true',
                           ])

py_mod = import('python')
py3 = py_mod.find_installation('python3')
py3_dep = py3.dependency()
message(py3.path())
message(py3.get_install_dir())

# Python Class Representation
py3.extension_module('pycart',
                    'vec.f90',
                    'pyclassderived.c',
                    dependencies : py3_dep,
                    install : true)

```

Usage

This can be built and used as:

```

meson setup bddir
meson compile -C bddir
python -c "import bddir.pycart as pycart; aak = pycart.pycart(1,10,2); print(aak); aak.unitstep(); print(aak)"
pycart(x: 1.000000, y: 10.000000, z: 2.000000)
pycart(x: 2.000000, y: 11.000000, z: 3.000000)

```

Conclusions

One of the best aspects of this approach is that the language pairs and representations line up nicely to prevent any extraneous copy operations. Representing the module subroutine as a class member function is a design choice unlikely to be made when generating bindings from `f2py`, but this was more for exploratory purposes.

To end on a rather provocative note [^fn:2], consider the timings for the wrapped derived type `pycart` and its equivalent pure-Python `fakecart` in Fig. 1 below:

Fortran wrapped cartesian derived type timings

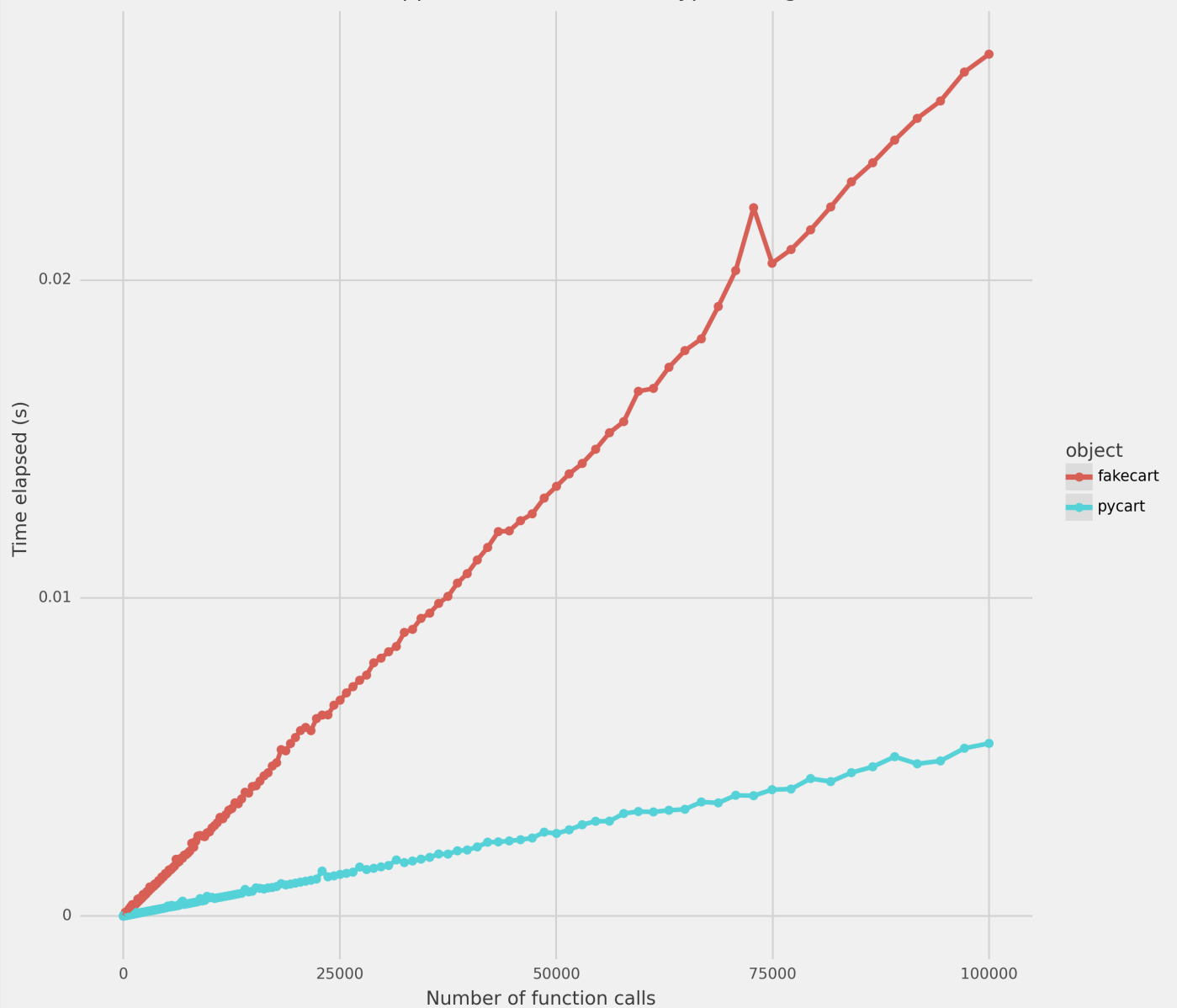


Figure 1: Results of comparing the pure-python class ('fakecart') to the C-extension class backed by a Fortran derived type ('pycart')

All the code for this post, along with other Fortran derived type representations is present in [this repository](#) under the examples folder.

Addendum and Future Directions

I had the pleasure of demonstrating part of this logic to fellow NumPy maintainers Dr. Pearu Peterson and Dr. Melissa Weber Mendonça and we were in agreement that the class strategy is the most flexible [^fn:3]. Pearu suggested using pointers passed between Fortran and C in an attempt to cover more than `bind(c)` compatible code, by generating simpler Fortran wrapper functions as well, and those will likely be among the next steps forward. The work of Pletzer et al. (2008) is also of interest, although their focus was on exposing a series of C functions which call Fortran methods.

References

Pletzer, Alexander, Douglas McCune, Stefan Muszala, Srinath Vadlamani, and Scott Kruger. 2008. "Exposing Fortran Derived Types to C and Other Languages." **Computing in Science Engineering** 10 (4): 86--92. <<https://doi.org/10.1109/MCSE.2008.94>>.

finicky](https://pythonspeed.com/articles/consistent-benchmarking-in-ci/), and no attempt was made to ensure statistical stability of the results nor were the plotted points obtained on machines dedicated and isolated from spurious loads

informally mentioned this during some LFortran meetings as well